Distributed Big Data Pipeline

Hadoop-HBase-Solr Integration for High-Throughput Search

Team Quintet & Sayon Mondal
Department of Computer Science and Engineering
Indian Institute of Technology Kharagpur

April 2026

Abstract

This report details the architecture, configuration, and implementation of a distributed, enterprise-grade data pipeline designed to ingest, process, and serve massive datasets with sub-second latency. Standard monolithic databases struggle with high concurrency and complex analytical queries. To resolve this, our system integrates Hadoop HDFS for distributed storage, MapReduce for parallel data processing, HBase for structured storage, and a SolrCloud cluster for in-memory, high-throughput search operations. A Next.js web application acts as the user interface, while system resilience is benchmarked using Siege.

Contents

1 Introduction **3**
1.1 The Challenge . . . 3
1.2 Engineering Objective . . . 3

2 System Architecture & Project Flow **3**

3 Data Storage & Processing Layer **4**
3.1 Hadoop Cluster Topology . . . 4
3.2 Hadoop HDFS Distribution . . . 4
3.3 MapReduce Execution . . . 4
3.3.1 Job 1 — Data Cleaning . . . 4
3.3.2 Job 2 — JSON Conversion . . . 5

4 High-Throughput Search Engine (SolrCloud) **5**
4.1 Cluster Topology & Networking . . . 5
4.2 Environment Optimization . . . 5
4.3 Sharding & Replication Strategy . . . 5
4.4 Strict Schema Definition & Ingestion . . . 6
4.5 Query Processing & Zero Network Hop . . . 6

5 Web Application Layer **6**

6 Stress Testing & Benchmarking **7**
6.1 Test Parameters 7
6.2 Benchmark Results 7

7 Conclusion & Future Scope **7**
7.1 System Achievements 7
7.2 Future Technical Scope 7

1 Introduction

1.1 The Challenge

Standard monolithic databases struggle to ingest massive datasets and serve complex analytical queries with sub-second latency under high concurrency. Processing a 1GB+ dataset like the IMDb movie repository requires an architecture that mitigates traditional disk-read bottlenecks and eliminates single points of failure.

1.2 Engineering Objective

Our engineering objective is to design a distributed pipeline capable of:

- Securely ingesting and processing a 1GB+ dataset.
- Distributing storage reliably across physical machines using a Tailscale VPN Mesh.
- Executing filtering and transformation via distributed MapReduce across a 3-node cluster.
- Executing filtering and sorting entirely in local RAM to guarantee sub-second query resolution.
- Providing a low-latency, modern web interface via Next.js.

2 System Architecture & Project Flow

The project follows a linear, highly decoupled flow of data from raw data files to a high-speed web interface.

1. **Raw Data Ingestion:** The system ingests a combined `combined.tsv` file merged from `title.basics.tsv` and `title.ratings.tsv`, totalling 1,658,156 records.
2. **Hadoop HDFS (Storage):** Raw data is uploaded to HDFS and distributed across the cluster with a replication factor of 3, split into 2 blocks across the 3-node cluster.
3. **MapReduce (Processing):** Two sequential MapReduce jobs execute distributed data cleaning and JSON transformation directly within HDFS. 1,658,156 raw records are reduced to 330,665 valid movie documents.
4. **SolrCloud (In-Memory Index):** Processed JSON documents are ingested into a distributed Solr cluster, which builds Inverted Indexes and columnar DocValues for rapid search capabilities.
5. **Next.js API & Frontend:** A backend Node.js server queries Solr via REST GET requests and serves the structured JSON to the Next.js frontend UI.
6. **Stress Testing:** The Next.js API endpoints are subjected to heavy concurrent traffic using the Siege benchmarking tool.

3 Data Storage & Processing Layer

3.1 Hadoop Cluster Topology

The Hadoop cluster consists of 3 physical nodes interconnected over a Tailscale VPN mesh operating on the `100.x.x.x` IP range. All inter-node communication — including HDFS block transfer and YARN resource management — is routed over Tailscale, providing secure connectivity without requiring nodes to be on the same physical network.

YARN (Yet Another Resource Negotiator) manages cluster resources and schedules distributed jobs. The ResourceManager allocates CPU and memory across nodes, while NodeManagers execute tasks in parallel using isolated containers.

- **Master Node (HvR-Asus, 100.96.118.120):** Runs the NameNode, SecondaryNameNode, ResourceManager, and NodeManager. Acts as the HDFS namespace authority and YARN job scheduler.
- **Worker 1 (jaypc / w1, 100.100.186.77):** Runs DataNode and NodeManager. Contributes 146.59 GB storage capacity.
- **Worker 2 (shresth-Inspiron-14-5410 / w2, 100.65.255.76):** Runs DataNode and NodeManager. Contributes 97.87 GB storage capacity.

Total configured HDFS capacity: **1.22 TB** across all 3 nodes.

3.2 Hadoop HDFS Distribution

The raw `combined.tsv` file is stored in HDFS under `/input/combined.tsv` and split into **2 blocks**, each replicated across all 3 DataNodes (replication factor = 3). This ensures fault tolerance — the dataset remains fully accessible even if one node goes offline. HDFS block placement is data-local, meaning MapReduce tasks are scheduled on the nodes physically holding each block, eliminating unnecessary network transfer during processing.

3.3 MapReduce Execution

MapReduce splits the HDFS input into chunks distributed across nodes. Each **Mapper** processes its local chunk independently and in parallel. The **Shuffle** phase sorts and groups intermediate key-value pairs by key across the network. **Reducers** then aggregate the grouped data into the final output written back to HDFS.

Two sequential MapReduce jobs were implemented to transform the raw dataset into Solr-ready JSON documents.

3.3.1 Job 1 — Data Cleaning

CleanMapper reads raw IMDB records from HDFS. For each row it drops records with missing critical fields (`tconst`, `originalTitle`, `startYear`), defaults missing numeric fields (`rating`, `votes`, `runtime`, `genres`) to `0` rather than discarding the record, discards unused columns (`isAdult`, `endYear`, `primaryTitle`), and emits only the 8 columns required by the Solr schema with the movie ID as the map key. **CleanReducer** acts as a passthrough, writing cleaned records directly to HDFS output.

3.3.2 Job 2 — JSON Conversion

JsonMapper reads the cleaned TSV output from Job 1. For each row it parses the 8-column TSV, converts the `genres` field from a comma-separated string into a JSON array (matching Solr's `multiValued` field type), casts numeric fields to correct types (`pint`, `pfloat`), sanitises title strings by removing escape characters, and emits a complete JSON document per record. **JsonReducer** acts as a passthrough, writing one JSON document per line to `/json_output` in HDFS.

A sample output document:

```
{"id":"tt0468569","title_type":"movie","title":"The Dark Knight",
 "genres":["Action","Crime","Drama"],"release_year":2008,
 "rating":9.0,"votes":2800000,"runtime":152}
```

Both jobs ran across all 3 nodes simultaneously with tasks scheduled data-locally, processing raw data in under 2 minutes total.

4 High-Throughput Search Engine (SolrCloud)

The core serving layer of the architecture is a highly optimized Apache Solr 9.10.1 cluster running on Ubuntu 24.04 (Noble Numbat) and Java 11.

4.1 Cluster Topology & Networking

The cluster consists of 1 Primary Node (hosting the embedded ZooKeeper registry on port 9983) and 3 Replica Nodes (Followers). All laptops in the cluster are securely tethered via a Tailscale VPN Mesh, operating on the `100.x.x.x` IP range. Custom hostnames (e.g., `solr`, `solr1`) are mapped in the `/etc/hosts` files to ensure stable inter-node communication.

4.2 Environment Optimization

To prevent "Connection Refused" errors and thread starvation during stress testing, OS file limits (`ulimits`) were elevated to 65,000 for both `nofile` and `nproc` on all nodes. Furthermore, the Solr JVM heap (`SOLR_HEAP`) was upgraded to 4GB to push the entire index into RAM, bypassing slow disk-read bottlenecks.

4.3 Sharding & Replication Strategy

The `imdb` collection was created using a zero-network-hop architecture:

- **Shards (1):** Prevents the dataset from being partitioned, entirely eliminating "Scatter-Gather" network latency.
- **Replication Factor (4):** Clones the exact dataset across all 4 laptops. Any node can independently process and answer a query in $O(1)$ time.

Field	Type	Indexed	DocValues
id	string	Yes	–
title_type	string	Yes	–
title	text_general	Yes	–
genres	strings (multiValued)	Yes	–
release_year	pint	Yes	Yes
rating	pfloat	Yes	Yes
votes	pint	Yes	Yes
runtime	pint	No	–

Table 1: Solr Schema Field Definitions

4.4 Strict Schema Definition & Ingestion

Data-driven schema guessing was disabled in favor of a strict schema pushed via the Solr REST API. The schema defines the following fields:

Numeric attributes (`rating`, `votes`, `release_year`) were configured with `docValues: true`, forcing Solr to build columnar memory structures for instantaneous range filtering and sorting. Data ingestion of JSON documents was executed via a batched hard-commit REST POST request (`commit=true`), forcing Solr to instantly build the Inverted Index and synchronise across all replicas over the Tailscale mesh.

4.5 Query Processing & Zero Network Hop

Queries arrive from the Next.js backend via HTTP GET requests. The cluster processes these utilizing localized Boolean intersections:

1. **Text Analysis:** Keyword matches (e.g., `q=title:Batman`) are evaluated against the Inverted Index in $O(1)$ time.

2. **Numeric Filtering:** Filter queries (e.g., `fq=release_year:[2000 TO 2010]`) and sorting scan the columnar DocValues.

3. **Response:** Solr serializes the intersecting Document IDs into a lightweight JSON array and transmits it back to the backend.

5 Web Application Layer

The user interface is powered by Next.js.

- **Backend API:** Acts as a middleware, receiving user inputs and translating them into optimized Solr REST GET queries.

- **Frontend UI:** Provides a modern, responsive interface for users to search, filter, and sort movies seamlessly.

6 Stress Testing & Benchmarking

To prove the architecture's resilience, we simulate heavy traffic against the API using the Siege tool.

6.1 Test Parameters

- **Load:** 200 concurrent user threads.
- **Duration:** 60-second sustained stress test.
- **Target:** Next.js endpoints performing complex Solr queries.

6.2 Benchmark Results

[To be filled in: Siege performance metrics including Total Transactions, Availability, Average Response Time, and QPS.]

7 Conclusion & Future Scope

7.1 System Achievements

The project successfully built an end-to-end distributed big data pipeline. A 3-node Hadoop cluster processed 1,658,156 raw IMDB records via two MapReduce jobs — reducing and transforming the dataset to clean JSON documents in under 2 minutes. These documents were ingested into a 4-node SolrCloud cluster delivering sub-second query responses, served through a Next.js web interface.

7.2 Future Technical Scope

Future enhancements include expanding the Tailscale mesh to integrate more replica nodes for increased throughput, and integrating Apache Hive to support complex, offline analytical queries that Solr is not naturally designed to execute.